CSE225L – Data Structures and Algorithms Lab
Lab 07
Sorted List (array based)

In today's lab we will design and implement the List ADT where the items in the list are sorted.

sortedtype.h

```
#ifndef SORTEDTYPE_H_INCLUDED
#define SORTEDTYPE_H_INCLUDED

const int MAX_ITEMS = 5;
template <class ItemType>
class SortedType
{
    public :
        SortedType();
        void MakeEmpty();
        bool IsFull();
        int LengthIs();
        void InsertItem(ItemType);
        void DeleteItem(ItemType);
        void RetrieveItem(ItemType&,
bool&);
        void ResetList();
        void GetNextItem(ItemType&);
    private:
        int length;
        ItemType info[MAX_ITEMS];
        int currentPos;
};
#endif // SORTEDTYPE_H_INCLUDED
```

sortedtype.cpp

```
#include "sortedtype.h"
template <class ItemType>
SortedType<ItemType>::SortedType()
{
    length = 0;
    currentPos = - 1;
}
template <class ItemType>
void SortedType<ItemType>::MakeEmpty()
{
    length = 0;
}
template <class ItemType>
bool SortedType<ItemType>::IsFull()
{
    return (length == MAX_ITEMS);
}
template <class ItemType>
int SortedType<ItemType>::LengthIs()
{
    return length;
}
template <class ItemType>
void SortedType<ItemType>::ResetList()
{
    currentPos = - 1;
}
template <class ItemType>
void
SortedType<ItemType>::GetNextItem(ItemType&
item)
{
    currentPos++;
    item = info [currentPos];
}
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType
item)
{
    int location = 0;
    bool moreToSearch = (location < length);

    while (moreToSearch)
    {
        if(item > info[location])
        {
            location++;
            moreToSearch = (location < length);
        }
        else if(item < info[location])
            moreToSearch = false;
    }
    for (int index = length; index > location;
index--)
        info[index] = info[index - 1];
    info[location] = item;
    length++;
}
template <class ItemType>
void SortedType<ItemType>::DeleteItem(ItemType
item)
{
    int location = 0;

    while (item != info[location])
        location++;
    for (int index = location + 1; index < length;
index++)
        info[index - 1] = info[index];
    length--;
}
template <class ItemType>
void SortedType<ItemType>::RetrieveItem(ItemType&
item, bool& found)
{
    int midPoint, first = 0, last = length - 1;
    bool moreToSearch = (first <= last);
    found = false;
    while (moreToSearch && !found)
    {
        midPoint = (first + last) / 2;
        if(item < info[midPoint])
        {
            last = midPoint - 1;
            moreToSearch = (first <= last);
        }
        else if(item > info[midPoint])
        {
            first = midPoint + 1;
            moreToSearch = (first <= last);
        }
        else
        {
            found = true;
            item = info[midPoint];
        }
    }
}
```

Generate the **driver file (main.cpp)** where you perform the following tasks. Note that you cannot make any change to the header file or the source file.

Operation to Be Tested and Description of Action	**Input Values**	**Expected Output**
• Create a list of integers		
• Print length of the list		0
• Insert five items	5 7 4 2 1	
• Print the list		1 2 4 5 7
• Retrieve 6 and print whether found		Item is not found
• Retrieve 5 and print whether found		Item is found
• Print if the list is full or not		List is full
• Delete 1		
• Print the list		2 4 5 7
• Print if the list is full or not		List is not full
• Write a class `timeStamp` that represents a time of the day. It must have variables to store the number of seconds, minutes and hours passed. It also must have a function to print all the values. You will also need to overload a few operators.		
• Create a list of objects of class `timeStamp`.		
• Insert 5 time values in the format ssmmhh	15 34 23 13 13 02 43 45 12 25 36 17 52 02 20	
• Delete the timestamp 25 36 17		
• Print the list		15:34:23 13:13:02 43:45:12 52:02:20

Solution:

1)TimeStamp.h
2)TimeStamp.cpp
3)main.cpp

Inlude the template datatypes explicitly at the end of Sortedtype.cpp. Also include the TimeStamp.h header file in your SortedType.cpp source file.

```
template class SortedType<int>;
template class SortedType<timeStamp>;
```

```h
#ifndef TIMESTAMP_H
#define TIMESTAMP_H

class timeStamp {
private:
    int seconds;
    int minutes;
    int hours;

public:
    // Default constructor (doesn't take any parameters)
    timeStamp();

    // Set time values (basic setters)
    void setTime(int hh, int mm, int ss);

    // Get time values (basic getters)
    int getSeconds();
    int getMinutes();
    int getHours();

    // Overload < operator for sorting
    bool operator<(timeStamp other);

    // Overload > operator for sorting
    bool operator>(timeStamp other);

    // Overload == operator for comparison
    bool operator==(timeStamp other);

    // Overload != operator for comparison
    bool operator!=(timeStamp other);

    // Function to print the timeStamp in hh:mm:ss format
    void print();
};

#endif // TIMESTAMP_H
```

```cpp
#include <iostream>
#include "TimeStamp.h"

using namespace std;

// Default constructor initializing to zero
timeStamp::timeStamp() {
    hours = 0;
    minutes = 0;
    seconds = 0;
}

// Set the time (hh:mm:ss)
void timeStamp::setTime(int hh, int mm, int ss) {
    hours = hh;
    minutes = mm;
    seconds = ss;
}

// Getters
int timeStamp::getSeconds() {
    return seconds;
}

int timeStamp::getMinutes() {
    return minutes;
}

int timeStamp::getHours() {
    return hours;
}

// Overload < operator for sorting
bool timeStamp::operator<(timeStamp other) {
    if (hours != other.getHours()) {
        return hours < other.getHours();
    }
    else if (minutes != other.getMinutes()) {
        return minutes < other.getMinutes();
    }
    else {
        return seconds < other.getSeconds();
    }
}

// Overload > operator for sorting
bool timeStamp::operator>(timeStamp other) {
    if (hours != other.getHours()) {
        return hours > other.getHours();
    }
    else if (minutes != other.getMinutes()) {
        return minutes > other.getMinutes();
    }
    else {
        return seconds > other.getSeconds();
    }
}

// Overload == operator for comparison
bool timeStamp::operator==(timeStamp other) {
    return (hours == other.getHours() && minutes == other.getMinutes() && seconds == other.getSeconds());
}

// Overload != operator for comparison
bool timeStamp::operator!=(timeStamp other) {
    return !(hours == other.getHours() && minutes == other.getMinutes() && seconds == other.getSeconds());
}

// Print time in hh:mm:ss format
void timeStamp::print() {
    cout << hours << ":" << minutes << ":" << seconds << endl;}
```

```cpp
#include <iostream>
#include "SortedType.h"
#include "TimeStamp.h"

using namespace std;

int main() {
    // Create a SortedType list of integers
    SortedType<int> intList;

    // Print the initial length of the list (should be 0)
    cout << "Length of list: " << intList.LengthIs() << endl;

    // Insert five integers: 5, 7, 4, 2, 1
    intList.InsertItem(5);
    intList.InsertItem(7);
    intList.InsertItem(4);
    intList.InsertItem(2);
    intList.InsertItem(1);

    // Print the list of integers
    cout << "List of integers: ";
    int item;
    intList.ResetList();
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
    }
    cout << endl;

    // Retrieve 6 and print whether found
    bool found;
    item = 6;
    intList.RetrieveItem(item, found);
    if (found) {
        cout << "Item " << item << " is found." << endl;
    } else {
        cout << "Item " << item << " is not found." << endl;
    }

    // Retrieve 5 and print whether found
    item = 5;
    intList.RetrieveItem(item, found);
    if (found) {
        cout << "Item " << item << " is found." << endl;
    } else {
        cout << "Item " << item << " is not found." << endl;
    }

    // Check if the list is full
    if (intList.IsFull()) {
        cout << "List is full." << endl;
    } else {
        cout << "List is not full." << endl;
    }

    // Delete item 1 from the list
    item = 1;
    intList.DeleteItem(item);

    // Print the list of integers after deletion
    cout << "List of integers after deletion: ";
    intList.ResetList();
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
```

```
    }

    // Check if the list is full again
    if (intList.IsFull()) {
        cout << "List is full." << endl;
    } else {
        cout << "List is not full." << endl;
    }

    cout<<endl;
    cout <<"============================================================" <<endl;
    cout<<" timeStamp class implementation starts from here "<<endl;
    cout <<"============================================================" <<endl;
    cout<<endl;

   SortedType<timeStamp> timeList;

    // Create timeStamp objects and set time values
    timeStamp t1, t2, t3, t4, t5;
    t1.setTime(15, 34, 23);
    t2.setTime(13, 13, 02);
    t3.setTime(43, 45, 12);
    t4.setTime(25, 36, 17);
    t5.setTime(52, 02, 20);

    // Insert timeStamp objects into the sorted list
    timeList.InsertItem(t1);
    timeList.InsertItem(t2);
    timeList.InsertItem(t3);
    timeList.InsertItem(t4);
    timeList.InsertItem(t5);

    // Delete the timeStamp 25:36:17
    timeList.DeleteItem(t4);

    // Print the list of timeStamps
    std::cout << "List of timeStamps:" << std::endl;
    timeStamp t;
    timeList.ResetList();
    for (int i = 0; i < timeList.LengthIs(); i++) {
        timeList.GetNextItem(t);
        t.print();
    }

    return 0;
}
```